back_populate_tables

Recursive relationship management for QuantDB leaf tables

PR #19 — F006 Specific Back Propagate

Troy Sincomb | SciCrunch/quantitative-db

The Problem

Complex Table Hierarchies

QuantDB stores neuroanatomical measurements across 14+ tables with strict foreign key dependencies.

Leaf tables like ValuesCat and ValuesQuant depend on nearly every other table in the schema.

Manual foreign key management is error-prone and unscalable.

Dependency Depth

7 parent tables for ValuesCat

6 parent tables for ValuesQuant

4 table tiers in the hierarchy

The 4-Tier Table Hierarchy

1. **Root Tables**

   Addresses, Aspects, Units, ControlledTerms, DescriptorsInst, Objects

2. **Intermediate**

   DescriptorsCat, DescriptorsQuant, ValuesInst, ObjDescInst, ObjDescCat, ObjDescQuant

3. **Leaf Tables**

   ValuesCat, ValuesQuant

4. **Self-Referencing**

   instance_parent, class_parent, aspect_parent, dataset_object

What Is back_populate_tables?

A recursive SQLAlchemy function that traverses MANYTOONE relationships, creates or resolves all parent records, and commits the final object with all FK references intact.

Recursive Traversal

Walks all MANYTOONE and MANYTOMANY relationships up the tree

Auto-Create Parents

If a parent record is missing, it's created on the fly

Deduplication

Checks for existing records before inserting via filter_by + unique constraints

Transaction Safety

Full rollback on IntegrityError — no partial state left behind

How It Works

1. **Inspect Relationships**
   Iterate through all MANYTOONE / MANYTOMANY relationships on the ORM mapper
2. **Resolve Parents**
   For each parent: query DB by all columns. If found, sync attributes. If missing, proceed to create.
3. **Recurse Upward**
   Call back_populate_tables() on each parent — ensuring grandparents exist first
4. **Commit or Merge**
   Check if the object itself exists. If new → db.add() + commit. If existing → db.merge() + sync.
5. **Error Handling**
   On IntegrityError → full rollback + re-raise. UUIDs are converted to strings for consistency.

Required Population Order

Root Tables	Intermediate	Leaf (back_populate)
Addresses Aspects Units ControlledTerms DescriptorsInst Objects	DescriptorsCat DescriptorsQuant ValuesInst ObjDescInst ObjDescCat ObjDescQuant	ValuesCat ValuesQuant Handled by back_populate_tables()

Only leaf tables use back_populate_tables — root and intermediate use get_or_create

Usage: Complete Ingestion Flow

```
# 1. Root tables (get_or_create)
addr = get_or_create(session, Addresses(addr_type='constant'))
unit = get_or_create(session, Units(label='mm', iri='...'))

# 2. Intermediate tables
desc_quant = get_or_create(session, DescriptorsQuant(
    unit=unit.id, aspect=aspect.id, domain=desc_inst.id))

# 3. Create leaf object + set relationships
values_quant = ValuesQuant(value=42.5, value_blob={...})
values_quant.descriptors_inst = desc_inst
values_quant.descriptors_quant = desc_quant
values_quant.obj_desc_quant = obj_desc_quant

# 4. Let back_populate_tables handle everything
result = back_populate_tables(session, values_quant)
```

get_or_create vs back_populate_tables

`get_or_create(session, obj)`

For root & intermediate tables

- Query by all columns → if exists, return it
- Fallback: query by unique constraints
- If not found → create, commit, return
- On IntegrityError → rollback, re-query

Handles: Addresses, Aspects, Units, ControlledTerms, DescriptorsInst, Objects, ObjDesc, ValuesInst*

`back_populate_tables(session, obj)`

For leaf tables only

- Recurse all MANYTOONE relationships
- Resolve or create each parent first
- Sync FK attributes from existing parents
- Commit final object with all refs intact

Handles: ValuesCat, ValuesQuant (7+ FK dependencies each)

Real-World Usage: F006 Ingestion Pipeline

```
# f006_ingest.py — fiber/fascicle measurements

# 1. Load YAML mappings → creates root descriptors
descriptors = create_all_descriptors_from_yaml(
  session, MAPPINGS_FILE) # Aspects, Units, DescInst

# 2. Create dataset + package Objects (get_or_create)
dataset = get_or_create_dynamic(session, Objects,
  {'id': DATASET_UUID, 'id_type': 'dataset'})

# 3. Loop CSV rows → create instances per subject/sample
for row in csv_reader:
  inst = get_or_create_dynamic(session, ValuesInst,
    {'type':'sample', 'id_formal':row['sam_id']})

  # 4. Leaf values → back_populate handles all FKs
  back_populate_with_dependencies(session,
    'values_quant',
    value=row['fiber_area'], desc_quant=dq, ...)
```

Testing: Validating Against Original ingest.py

Dual-Pipeline Strategy

Two independent implementations:

quantdb/ingest.py
Original pipeline — manual FK wiring, calls ingest_fasc_fib() to fetch and insert

ingestion/f006_ingest.py
New pipeline — uses back_populate_tables / back_populate_with_dependencies for leaf inserts

Both must produce identical databases when given the same F006 dataset input.

What Gets Compared

Check	Method
Row counts per table	COUNT(*) across 10 tables
Aspects / Units / Terms	label + iri equality
DescriptorsInst	label + iri equality
Objects (dataset/pkg)	id + id_type equality
ValuesInst	id_formal + type + id_sub
ValuesQuant	value + orig_value

Test Suite Overview

`test_back_populate_simple.py`

Unit tests — minimal objects. Creates one ValuesCat and one ValuesQuant with minimal parents, verifies back_populate resolves all FKs.

`test_back_populate_tables.py`

Integration tests — full hierarchy. Builds all 4 tiers (root → leaf), tests missing-parent creation, documents population order.

`test_f006_database_comparison.py`

End-to-end comparison. Runs both ingest.py and f006_ingest.py, compares row counts + sample data across 10 tables.

Comparison Workflow

= compare row counts + sample data =

Clear DB → Run ingest.py → Snapshot | Clear DB → Run f006_ingest → Snapshot

F006 Dataset Application

What F006 Ingests

Neuroanatomical measurements from micro-CT imaging including:

- Nerve volumes (mm^3)
- Cross-section diameters (µm)
- Fiber-fascicle hierarchies
- Subject/sample metadata
- InterLex term mappings

How back_populate Helps

The F006 ingestion pipeline uses back_populate_tables for:

- ValuesCat — modality tags (micro-CT)
- ValuesQuant — actual measurements

This avoids manually wiring 7+ FK relationships per measurement row.

Best Practices & Pitfalls

Do

- Follow Root → Intermediate → Leaf order
- Set all relationship properties before calling back_populate_tables
- Use get_or_create for root and intermediate tables
- Match ID patterns: sub- for subjects, sam- for samples
- Always set id_file on package-type Objects

Don't

- Skip the population order — FK violations guaranteed
- Use dataset objects in obj_desc_* tables (only packages)
- Manually manage leaf table FKs — let back_populate handle it
- Forget required enum values (address_type, instance_type, etc.)
- Assume circular deps will resolve — they won't without recursion

Key Takeaways

back_populate_tables recursively resolves all parent relationships for leaf tables

Always follow Root → Intermediate → Leaf population order

Use get_or_create for early tiers, back_populate for leaf tables only

github.com/SciCrunch/quantitative-db/pull/19